FULL STACK LAB

LAB SHEET 13 : BOOKNEST LIBRARY TRACKER

Name: Shreya Choudhary
Course: B.Tech CSE – 3rd Year
Semester: 5th Semester
Roll No.: 61
Section: A
Course Name: Full Stack
Course Code: BTCS303P
Date: 14/10/2026

Problem Statement

Build “BookNest” — a secure full-stack library tracker that allows a librarian to manage books and issue/return operations, while members can search the catalogue and view only their own issued books. The system uses a 14-day loan period, a maximum of three active issues per member, a fine of ₹2 per full overdue day, hashed passwords, parameterised queries, automated tests and container deployment.

1. Repository Setup and Architecture

Create the repository 61_BookNest with /server, /client, README.md, queries.sql, tests, Dockerfile and .github/workflows/ci.yml. Architecture: Browser → React (Vite) → Express REST API → SQLite database. On login, the API verifies the bcrypt password hash and returns a JWT that expires in one hour. Protected requests send Authorization: Bearer <token>.

ER design: members(id, name, email UNIQUE, password_hash, role); books(id, code UNIQUE, title, author, total_copies, available_copies); issues(id, member_id FK, book_id FK, issued_on, due_on, returned_on, fine). The index issues(member_id, returned_on) speeds up active-issue lookup and a member’s issue-history query.

2. Database Design

```
CREATE TABLE members (
 id INTEGER PRIMARY KEY, name TEXT NOT NULL,
 email TEXT NOT NULL UNIQUE, password_hash TEXT NOT NULL,
 role TEXT NOT NULL CHECK(role IN ('librarian','member'))
);
CREATE TABLE books (
 id INTEGER PRIMARY KEY, code TEXT NOT NULL UNIQUE,
 title TEXT NOT NULL, author TEXT NOT NULL,
 total_copies INTEGER NOT NULL CHECK(total_copies >= 0),
 available_copies INTEGER NOT NULL CHECK(
  available_copies >= 0 AND available_copies <= total_copies)
);
CREATE TABLE issues (
 id INTEGER PRIMARY KEY, member_id INTEGER NOT NULL REFERENCES members(id),
 book_id INTEGER NOT NULL REFERENCES books(id),
 issued_on TEXT NOT NULL, due_on TEXT NOT NULL,
 returned_on TEXT, fine INTEGER NOT NULL DEFAULT 0
);
CREATE INDEX idx_issues_member_returned ON issues(member_id, returned_on);
```

Seed the four users using bcrypt hashes of Pass@123 and add the five books from the question. Seed issue #1 for Rohan and Database System Concepts, issued 20 days ago and due six days ago.

3. Database Queries

```
-- Q1: overdue list and fine (SQLite)
SELECT m.name, b.title,
 CAST(julianday('now')-julianday(i.due_on) AS INTEGER) AS days_overdue,
 CAST(julianday('now')-julianday(i.due_on) AS INTEGER)*2 AS fine
FROM issues i JOIN members m ON m.id=i.member_id
```

```
JOIN books b ON b.id=i.book_id
WHERE i.returned_on IS NULL AND i.due_on < date('now');

-- Q2: top 3 most-issued books
SELECT b.title, COUNT(i.id) AS issue_count
FROM books b LEFT JOIN issues i ON i.book_id=b.id
GROUP BY b.id,b.title ORDER BY issue_count DESC LIMIT 3;
```

4. Back-End REST API

Implement POST /api/auth/login; GET /api/books?search=&page=&limit=; POST /api/books; POST /api/issues; POST /api/issues/:id/return; GET /api/issues/overdue; and GET /api/issues/mine. Use dotenv, helmet, cors, request logging, routes/controllers/middleware/db folders and a central error handler returning { error: { code, message } } without stack traces.

For issuing a book, validate the request and check that the member exists and has fewer than three active issues. In one transaction run UPDATE books SET available_copies = available_copies - 1 WHERE id = ? AND available_copies > 0; verify one row changed, then insert the issue with due date = issue date + 14 days. Return 409 NO_COPIES or LIMIT_REACHED when applicable.

For returns, reject an already-returned issue with 409 ALREADY_RETURNED. Calculate fine = ₹2 × full days overdue (minimum ₹0), set returned_on and fine, and increment available copies in one transaction. All queries must be parameterised.

5. Front End (React)

Create a validated login form with server error messages and logout. Protected routes redirect unauthenticated users to /login and hide librarian-only pages from members. The books page provides case-insensitive search debounced by 300 ms, Previous/Next pagination from API metadata and available-copy counts. Librarians get an issue form, overdue table and Return button showing the fine; members get a My Issues list. Show loading indicators, error banners and empty states, support 360 px width, and centralise HTTP calls in api.js to attach the token and log out on 401.

6. Testing and Delivery

Use a separate test database reset before each test. Required tests: wrong password → 401; issuing the last copy twice → second request 409 NO_COPIES; fourth active issue → 409 LIMIT_REACHED; returning the overdue seed issue → fine ₹12; calculateFine on time → ₹0, one day late → ₹2 and ten days late → ₹20.

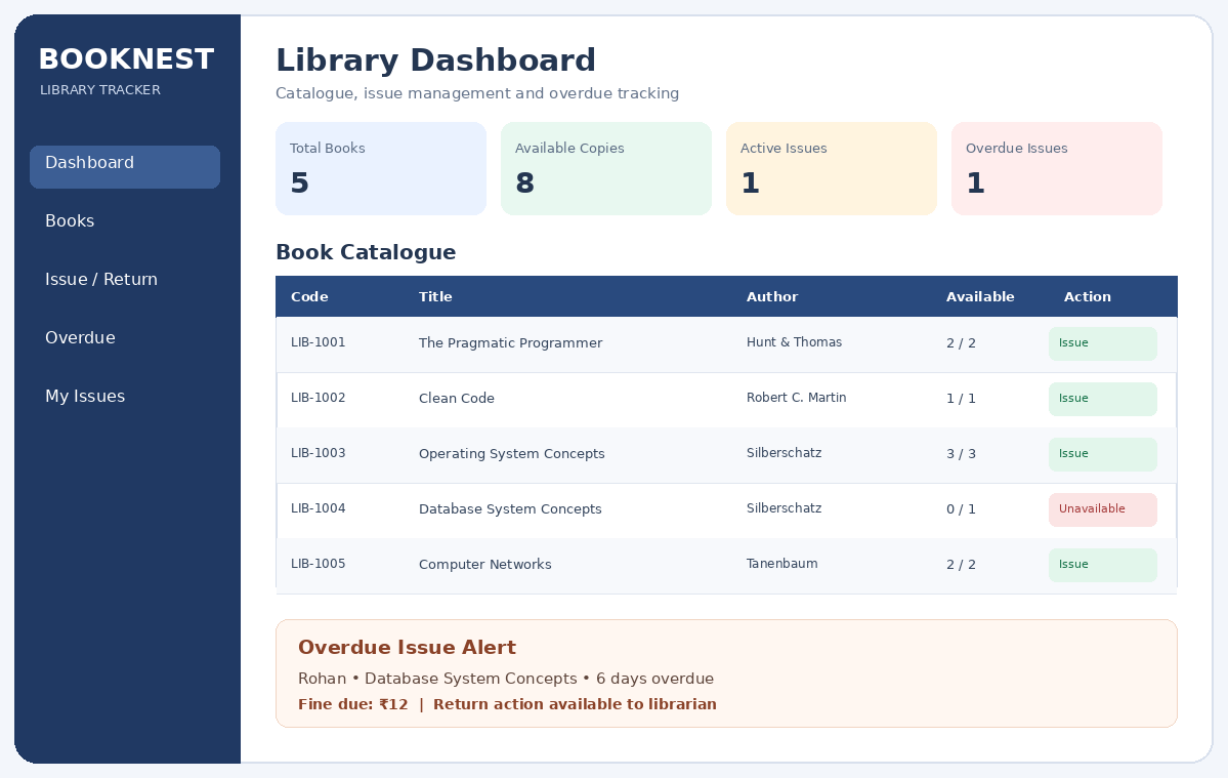
Add .env.example (PORT, JWT_SECRET, DB path), .gitignore excluding node_modules and .env, a Node 20 Alpine Dockerfile that runs as a non-root user, and GitHub Actions to run npm ci and npm test. README should include install, seed and run commands. Make at least six meaningful Git commits.

7. Examiner Verification

Verify: valid admin login returns a token and a wrong password returns 401; books without a token returns 401; page 2 with limit 2 returns books 3 and 4; search for silberschatz returns books 3 and 4; SQL-injection-like search is treated as plain text; issue book 2 to Aarav succeeds but issuing it to Meera returns 409 NO_COPIES; the overdue list shows Rohan with six days and ₹12; a second return returns 409 ALREADY_RETURNED; empty issue body returns 400; and two simultaneous requests for the last copy yield exactly one 201 and one 409.

Output:

BookNest Library Tracker — Librarian Dashboard



The preview shows the book catalogue, available copies, librarian actions and Rohan’s overdue issue with a ₹12 fine. This is a visual mock-up; live behaviour must be verified by running the application and tests.